

StreamReader and StreamWriter in C# with Examples

In this article, I am going to discuss **StreamReader and StreamWriter in C#** with Examples. Please read our previous article where we discussed [FileStream Class in C#](#) with Examples. At the end of this article, you will understand what StreamReader and StreamWriter are in C# and when and how to use StreamReader and StreamWriter in C# with Examples.

StreamWriter Class in C#

The StreamWriter Class in C# is more popular in File Handling and it is very helpful in writing text data into a file. It is easy to use and provides a complete set of constructors and methods to work on it. StreamWriter class in C# is used for writing characters to stream in a particular format. If you go to the definition of StreamWriter class then you will see the following. The StreamWriter Class in C# belongs to the **System.IO** namespace and implements the abstract **TextWriter** class.

```
namespace System.IO
{
    ...public class StreamWriter : TextWriter
    {
        ...public static readonly StreamWriter Null;

        ...public StreamWriter(Stream stream);
        ...public StreamWriter(string path);
        ...public StreamWriter(Stream stream, Encoding encoding);
        ...public StreamWriter(string path, bool append);
        ...public StreamWriter(Stream stream, Encoding encoding, int bufferSize);
        ...public StreamWriter(string path, bool append, Encoding encoding);
        ...public StreamWriter(Stream stream, Encoding encoding, int bufferSize, bool leaveOpen);
        ...public StreamWriter(string path, bool append, Encoding encoding, int bufferSize);

        ...public virtual bool AutoFlush { get; set; }
        ...public virtual Stream BaseStream { get; }
        ...public override Encoding Encoding { get; }

        ...public override void Close();
        ...public override void Flush();
        ...public override Task FlushAsync();
        ...public override void Write(char value);
        ...public override void Write(char[] buffer, int index, int count);
        ...public override void Write(string value);
        ...public override void Write(char[] buffer);
        ...public override Task WriteAsync(char value);
        ...public override Task WriteAsync(string value);
        ...public override Task WriteAsync(char[] buffer, int index, int count);
        ...public override Task WriteLineAsync();
        ...public override Task WriteLineAsync(char value);
        ...public override Task WriteLineAsync(string value);
        ...public override Task WriteLineAsync(char[] buffer, int index, int count);
        ...protected override void Dispose(bool disposing);
    }
}
```

As you can see in the above image, this class contains lots of methods, different types of constructors, and a few properties.

Constructor:

The Constructor is used to initialize a new instance of the **System.IO.StreamWriter** class for the specified file on the specified path, Specified Stream, specified encoding and buffer size. It has also different overloaded versions for different ways to create an instance of StreamWriter Class.

Methods:

1. **Close():** This method closes the current StreamWriter object and the underlying stream.
2. **Flush():** This method Clears data from all buffers for the current writer and causes any buffered data to be written to the underlying stream.
3. **Write():** It Writes data to the stream. It has different overloads for different data types to write in the stream.
4. **WriteLine:** It is the same as Write() but it adds the newline character at the end of the data. It has different overloads for different data types to write in the stream.
5. **Dispose():** It releases the unmanaged resources used by the StreamWriter and optionally releases the managed resources.

Properties:

1. **AutoFlush:** Gets or sets a value indicating whether the StreamWriter will flush its buffer to the underlying stream after every call to System.IO.StreamWriter.Write(System.Char).
2. **BaseStream:** Gets the underlying stream that interfaces with a backing store.
3. **Encoding:** Gets the System.Text.Encoding in which the output is written.

Example to Write User Input to a File using StreamWriter Class in C#:

It is very easy to write data into a text file using StreamWriter Class and most beginners prefer to use this class in writing files. In the below example, we are using the StreamWriter constructor (**public StreamWriter(string path);**) which takes the string path as an argument to create an instance of StreamWriter Class. This StreamWriter instance will create a file with the name **MyFile.txt** at the specified location i.e. in the **D Drive**. Using the **Console.ReadLine()** method we are taking the input data from the user which we will store in our **MyFile.txt** file. When we call the Write method using the StreamWriter object by passing the string data, it will write the string data into the stream i.e. into the text file. Finally, we call the Flush and Close method to clear all the buffers as well as close the stream. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Create an Instance of StreamWriter by specifying the String Pat
```

```

        // This will create a file with the name MyFile.txt at the specific
i.e. in the D Drive
        // Here we are using the StreamWriter constructor which takes the
as an argument to create an instance of StreamWriter class
        StreamWriter streamWriter = new StreamWriter("D://MyFile.txt");
        // Asking the user to enter the text that we want to write into the
MyFile.txt file
        Console.WriteLine("Enter the Text that you want to write on File");
        // To read the input from the user
        string inputData = Console.ReadLine();
        // To write the data into the stream use the Write Method of the StreamWriter
Object
        streamWriter.Write(inputData);
        Console.WriteLine("Data Has Been Written to the File");
        // Clears all the buffers for the current writer by calling the Flush
of the StreamWriter Object
        streamWriter.Flush();
        // Close the current StreamWriter object and the underlying stream
the Flush Method of the StreamWriter Object
        streamWriter.Close();
        Console.ReadKey();
    }
}
}

```

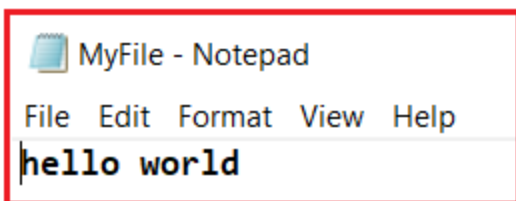
Output:

```

Enter the Text that you want to write on File
hello world
Data Has Been Written to the File

```

Now, you will see that, it will create a text file with the name **MyFile.txt** in the **D drive** and once you open the file you will see the following data written inside it.



Save Variable Data to File in C# using StreamWriter Class

Several times in Real-Time Applications, you need to save variable data in a file. These variable data might be the output of our program, Log details, Exceptions, Errors, Input Request data for your API, etc. Now, let us see, how we can save variable data in a file using StreamWriter Class in C#. For a better understanding, please have a look at the following Example. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;

```

```

using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set the file Path where you want to Create the File
            string filePath = @"D:\MyFile.txt";
            //Do Some Basic Operations using variables
            int a, b, result;
            a = 15;
            b = 20;
            result = a + b;
            //Now, we want to store the variable a, b, and result in the File
            //Creating the StreamWriter Object inside the using block by using
            Constructor which takes the FilePath
            //Using block automatically calls the Flush and Close Method of St
            Object

            using (StreamWriter streamWriter = new StreamWriter(filePath))
            {
                streamWriter.Write($"Sum of {a} + {b} = {result}");
                //No need to call the Flush and Close Method
            }
            Console.WriteLine("Variable Data is Saved into the File");
            Console.ReadKey();
        }
    }
}

```

Now open the **D:\MyFile.txt** file and you will see the following text.

Sum of 15 + 20 = 25

Now, if you run the application multiple times, then you will observe one thing, every time you run the application, the StreamWriter overwrites the content in the text file. Instead of overwriting the content, we want to append the content to the text file. If you want to do so, then you need to use the other overloaded version of the StreamWriter constructor which should take the boolean Append parameter and if you need to pass the value true.

For a better understanding, please have a look at the following example. This is the same example as the previous one. Here, the only changes we have done are, using the other overloaded version of the StreamWriter constructor which takes string path and boolean append parameter and instead of the Write method we are using WriteLine method of the StreamWriter object.

```

using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Set the file Path where you want to Create the File
        string filePath = @"D:\MyFile.txt";
        //Do Some Basic Operations using variables
        int a, b, result;
        a = 15;
        b = 20;
        result = a + b;
        //Now, we want to store the variable a, b, and result in the File
        //Creating the StreamWriter Object inside the using block by using
        Constructor which takes the FilePath
        //Using block automatically calls the Flush and Close Method of StreamWriter
        Object
        //Passing the boolean true for Append which means it will not overwrite
        instead it will append the content
        using (StreamWriter streamWriter = new StreamWriter(filePath, true))
        {
            streamWriter.WriteLine($"Sum of {a} + {b} = {result}");
            //No need to call the Flush and Close Method
        }
        Console.WriteLine("Variable Data is Saved into the File");
        Console.ReadKey();
    }
}

```

Now, with the above changes in place, run the application multiple times, and you will see, the content is going to be appended.

StreamReader Class in C#

The StreamReader class in C# allows us to read text files. Its implementation is easy and it is widely popular among developers. However, there are many ways to read text files in C# but StreamReader Class is more popular in the list. While working with C# StreamReader Class, you need to remember the following points.

1. Implements a TextReader that reads characters from a byte stream in a particular encoding.
2. StreamReader class uses UTF-8 Encoding by default.
3. StreamReader class is designed for character input in a particular encoding.
4. Use this class for reading a standard text file.
5. By default, it is not thread-safe.

If you go to the definition of StreamReader class then you will see the following. The StreamReader class belongs to the **System.IO** namespace and implements the abstract **TextReader** class. The StreamReader class in C# is used for reading characters from the stream in a particular format.

```

namespace System.IO
{
    ...public class StreamReader : TextReader
    {
        ...public static readonly StreamReader Null;

        ...public StreamReader(Stream stream);
        ...public StreamReader(string path);
        ...public StreamReader(Stream stream, bool detectEncodingFromByteOrderMarks);
        ...public StreamReader(Stream stream, Encoding encoding);
        ...public StreamReader(string path, bool detectEncodingFromByteOrderMarks);
        ...public StreamReader(string path, Encoding encoding);
        ...public StreamReader(Stream stream, Encoding encoding, bool detectEncodingFromByteOrderMarks);
        ...public StreamReader(string path, Encoding encoding, bool detectEncodingFromByteOrderMarks);
        ...public StreamReader(Stream stream, Encoding encoding, bool detectEncodingFromByteOrderMarks, int bufferSize);
        ...public StreamReader(string path, Encoding encoding, bool detectEncodingFromByteOrderMarks, int bufferSize);
        ...public StreamReader(Stream stream, Encoding encoding, bool detectEncodingFromByteOrderMarks, int bufferSize, bool leaveOpen);

        ...public virtual Encoding CurrentEncoding { get; }
        ...public bool EndOfStream { get; }
        ...public virtual Stream BaseStream { get; }

        ...public override void Close();
        ...public void DiscardBufferedData();
        ...public override int Peek();
        ...public override int Read();
        ...public override int Read(char[] buffer, int index, int count);
        ...public override Task<int> ReadAsync(char[] buffer, int index, int count);
        ...public override int ReadBlock(char[] buffer, int index, int count);
        ...public override Task<int> ReadBlockAsync(char[] buffer, int index, int count);
        ...public override string ReadLine();
        ...public override Task<string> ReadLineAsync();
        ...public override string ReadToEnd();
        ...public override Task<string> ReadToEndAsync();
        ...protected override void Dispose(bool disposing);
    }
}

```

As you can see in the above image, the StreamReader class contains lots of methods, different types of constructors, and a few properties.

Constructor:

It initializes a new instance of the StreamReader class for the specified stream or string path based on the specified character encoding, byte order mark detection option, and buffer size, and optionally leaves the stream open. It has different overloaded versions which you can see in the above image.

Methods:

1. **Close():** The Close method Closes the StreamReader object and the underlying stream, and releases any system resources associated with the reader.
2. **Peek():** This method returns the next available character but does not consume it. An integer represents the next character to be read, or -1 if there are no characters to be read or if the stream does not support seeking.
3. **Read():** This method reads the next character from the input stream and advances the character's position by one character. The next character from the input stream is represented as a System.Int32 object, or -1 if no more characters are available.
4. **ReadLine():** This method Reads a line of characters from the current stream and returns the data as a string. The next line from the input stream, or null if the end of the input stream is reached.
5. **Seek():** It is used to read/write at a specific location from a file.

Properties:

1. **CurrentEncoding:** It gets the current character encoding that the current System.IO.StreamReader object is using.
2. **EndOfStream:** It gets a value that indicates whether the current stream position is at the end of the stream.
3. **BaseStream:** It returns the underlying stream.

Example to Read Data from a File using StreamReader Class in C#:

In the below example first, we create the File Path variable which is holding the path of the existing File where our data is saved. Then we create an instance of StreamReader by passing the file Path variable to read the data from the file. Then we specify from where to start reading the input stream by calling the Seek method of the underlying stream of the StreamReader object. Then we are using the ReadLine method of the StreamReader object to read lines of characters from the stream and we use a while loop to read all lines of characters from the stream. Finally, we close the stream by calling the Close method of the StreamReader Object. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create a Variable to Hold the String Path
            string filePath = "D://MyFile.txt";
            //Creating an Instance StreamReader Object to Read the Data from 1
Path
            StreamReader streamReader = new StreamReader(filePath);
            Console.WriteLine("Content of the File");
            // This is used to specify from where to start reading the input :
            // BaseStream: Returns the underlying stream.
            // Seek: The new position within the current stream.
            // SeekOrigin.Begi: Specifies the beginning of a stream
            streamReader.BaseStream.Seek(0, SeekOrigin.Begin);
            //It reads a line of characters from the current stream and return
as a string.
            //It return the next line from the input stream, or null if the en
input stream is reached.
            string strData = streamReader.ReadLine();
            // To Read the whole file line by line use While Loop as long the
not null
            while (strData != null)
            {
                //Print the String data
                Console.WriteLine(strData);
                //Then Read the next String data
                strData = streamReader.ReadLine();
            }
            Console.ReadLine();
        }
    }
}
```



```

        //Close the StreamReader Object
        streamReader.Close();
        Console.ReadKey();
    }
}

```

Output:

```

Content of the File
Sum of 15 + 20 = 25

```

Reading Data using the ReadToEnd method of StreamReader Object:

Instead of using the ReadLine method, we can also use the ReadToEnd method to read all the contents of the text file. For a better understanding, please have a look at the following example. Here, instead of using the ReadLine method, we are using the ReadToEnd method of the StreamReader object.

```

using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create a Variable to Hold the String Path
            string filePath = "D://MyFile.txt";
            //Creating an Instance StreamReader Object to Read the Data from 1
            Path
            StreamReader streamReader = new StreamReader(filePath);
            Console.WriteLine("Content of the File");
            //Reading text file using StreamReader Class
            using (StreamReader reader = new StreamReader(filePath))
            {
                Console.WriteLine(reader.ReadToEnd());
            }

            Console.ReadKey();
        }
    }
}

```

Output:

```

Content of the File
Sum of 15 + 20 = 25

```


Using both StreamReader and StreamWriter Class in C#

As discussed, it is very easy to read a text file using StreamReader Class in C#. In the below example, we are doing the following thing:

- Write some data on a text file using StreamWriter Class
- Read those data using the StreamReader Class.

In the below example, we are using both StreamWriter and StreamReader Class to perform write and read operations on a file. Here, I am showing two approaches to read the data from a text file i.e. using the StreamReader **ReadToEnd** method as well as the **ReadLine** method. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace FileHandlinDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Set The File Path
            string filePath = @"D:\MyTextFile.txt";
            //Write Some data to a text file using StreamWriter
            using (StreamWriter streamWriter = new StreamWriter(filePath))
            {
                streamWriter.WriteLine("Welcome to DotNetTutorials");
                streamWriter.WriteLine("You are Learning File Handling in C#");
            }
            //Reading text file using StreamReader Class ReadToEnd Method
            Console.WriteLine("\nReading Approach 1: using ReadToEnd Method");
            using (StreamReader reader = new StreamReader(filePath))
            {
                Console.WriteLine(reader.ReadToEnd());
            }
            //Another Approach to Read the Data from a text file using StreamReader
            Console.WriteLine("\nReading Approach 2: using ReadLine Method");
            StreamReader streamReader = new StreamReader(filePath);
            streamReader.BaseStream.Seek(0, SeekOrigin.Begin);
            string strData = streamReader.ReadLine();
            while (strData != null)
            {
                Console.WriteLine(strData);
                strData = streamReader.ReadLine();
            }
            Console.ReadKey();
        }
    }
}
```

```
}
```

Output:

```
Reading Approach 1: using ReadToEnd Method
Welcome to DotNetTutorials
You are Learning File Handling in C#

Reading Approach 2: using ReadLine Method
Welcome to DotNetTutorials
You are Learning File Handling in C#
```

In the next article, I am going to discuss [File Class in C#](#) with Examples. Here, in this article, I try to explain **StreamReader and StreamWriter in C#** with Examples. I hope you enjoy this StreamReader and StreamWriter in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.